Plan: Personalized Travel Planning Chatbot

- The user interacts with the chatbot to receive a fully personalized travel plan based on preferences such as budget, accommodation type, food restrictions, and activity interests.

2. User Input (Structured + Natural Language)

2.1 Core Inputs

- **Travel Information**
  - Start point
  - Destination (e.g. Paris, Berlin, Tokyo)
  - Number of People
  - Time
- **Budget**
  - tight (low budget)
  - medium
  - Luxury
- **Accommodation Preferences**
  - hostel
  - 3-star hotel
  - 4-star hotel
  - 5-star hotel
- **Restaurant Preferences**
- **Food Type**
  - local cuisine
  - vegetarian / vegan
  - international
- **Allergies / Restrictions**
  - gluten-free
  - lactose-free
  - nut allergy
- **Activity Preferences**
- **Activity Type**
  - city tour
  - cultural (museum, history)
  - adventure (dangerous activities)
  - water activities

    - sky activities (e.g. skydiving)
- **Traveling style:**
    - budget traveler
    - luxury traveler
    - adventure traveler
    - cultural explorer
    - foodie traveler
    - relaxation / leisure traveler
    - family-friendly traveler

2.2 Example Input: "From Munich, I want to travel to Barcelona with 2 people for 2 days, medium budget, 4-star hotel, vegetarian food, and I like water activities and city tours."

Or the ChatBot asks each question, and the user types in

3. System Output (Generated by Chatbot): The chatbot generates a structured and personalized travel recommendation.

- Hotel Recommendations. Output includes:
    - Hotel name, price range, rating, short description
- Restaurant Recommendations. Output includes:
    - restaurant name
    - cuisine type
    - special notes (e.g. "vegan-friendly")
- Activity Recommendations
- Transportation Recommendations. Output includes:
    - Travel to Destination:
        - Flights
        - Trains
        - Bus
    - Transportation Inside City:
        - public transport
        - taxi / ride-sharing
        - walking suggestions
- Itinerary Generation: day-by-day travel plan (morning / afternoon / evening activities)

4. Adaptive and Context-Aware Travel Planning

The system dynamically updates recommendations based on user behavior and context

- History (event-based):
  User inputs where they have been (e.g. museum, restaurant) and duration
- Current State (feeling):
  User-selected or inferred (e.g. tired, hungry, energetic)
- Preferences:
  Food type, activity interests, travel style
- Context:
  Current location (approx.), time, weather

-> Next Recommendation = f(history, current state, preferences, location, time) (this will be processed by GeminiAI model)

USE CASE

UC1 – Generate Personalized Travel Plan

Actors — User (primary), AI chatbot (Gemini), recommendation system
Goal — Generate a complete personalized travel plan
Preconditions — User has provided travel inputs (destination, budget, preferences)

Basic flow —
User opens chatbot → enters travel details (structured or natural language) → system processes input → AI generates recommendations (hotel, restaurant, activities, transport) → system displays full itinerary

Alternative flows —
User provides incomplete input → chatbot asks follow-up questions;
AI output does not meet constraints → system regenerates plan

---

UC2 – Get Hotel Recommendations

Actors — User, AI chatbot
Goal — Receive suitable hotel options

Preconditions — Destination and accommodation preferences are defined

Basic flow —
User requests hotels → system filters based on budget + hotel type → AI generates list → system displays name, price, rating, description

Alternative flows —
No matching hotels → system suggests alternatives (e.g. lower star level or higher budget)

UC3 – Get Restaurant Recommendations

Actors — User, AI chatbot
Goal — Receive suitable restaurant options

Preconditions — Food preferences or restrictions are defined

Basic flow —
User requests restaurants → system filters based on food type and allergies → AI generates recommendations → system displays cuisine type and notes

Alternative flows —
Conflicting preferences (e.g. vegan + seafood) → system asks for clarification

---

UC4 – Get Activity Recommendations

Actors — User, AI chatbot
Goal — Receive relevant activities

Preconditions — Activity preferences are defined

Basic flow —
User requests activities → system evaluates preferences (e.g. cultural, adventure) → AI generates suggestions → system displays activities

Alternative flows —
Too many options → system narrows results based on time or location

---

UC5 – Get Transportation Plan

Actors — User, AI chatbot
Goal — Plan travel and local transport

Preconditions — Start point and destination are defined

Basic flow —
User requests transport → system evaluates options (flight/train/bus) → AI generates recommendations → system displays transport plan

Alternative flows —
No direct route → system suggests alternative routes

UC6 – Generate Itinerary

Actors — User, AI chatbot
Goal — Get day-by-day travel plan

Preconditions — Travel duration and preferences are defined

Basic flow —
User requests itinerary → AI organizes activities into time slots → system displays structured plan (morning/afternoon/evening)

Alternative flows —
Too many activities → system reduces or redistributes

UC7 – Modify Travel Plan

Actors — User, AI chatbot
Goal — Update existing travel plan

Preconditions — A travel plan already exists

Basic flow —
User changes input (e.g. budget, hotel type) → system updates constraints → AI regenerates plan → system displays updated version

Alternative flows —
New constraints conflict → system suggests adjustments

UC8 – Provide Context-Aware Recommendation (Core AI)

Actors — User, AI chatbot (Gemini), context engine
Goal — Recommend next activity dynamically

Preconditions —
User has:

- **history (visited places)**
- **current state (feeling)**
- **preferences**
- **location/time**

Basic flow —
User inputs visited place + duration → system updates history → user selects or system infers current state → system builds context → AI computes next recommendation → system displays next best activity

Alternative flows —
User does not provide state → system infers from history;
Insufficient context → system asks user for clarification

UC9 – Update Recommendation Based on Behavior

Actors — User, AI chatbot
Goal — Adapt recommendations dynamically

Preconditions — Ongoing travel session

Basic flow —
User skips or changes activity → system updates history → AI recalculates next step → system displays updated recommendation

Alternative flows —
User ignores recommendations → system adjusts strategy (e.g. simpler suggestions)

UC10 – Handle Conflicting Preferences

Actors — User, AI chatbot
Goal — Resolve input conflicts

Preconditions — User provides inconsistent inputs

Basic flow —
System detects conflict → AI explains issue → system suggests alternatives → user selects option

Alternative flows —
User insists → system prioritizes selected constraint

UC11 – Explore Alternative Plans

Actors — User, AI chatbot
Goal — Compare multiple travel options

Preconditions — Base input exists

Basic flow —
User requests alternatives → AI generates multiple plans (e.g. relaxed vs adventure) → system displays comparison

Alternative flows —
User refines one option → system updates selected plan

Tech stack:

- Frontend: ReactJS
- Frontend Framework: NextJS
- Backend Framework: NestJS with Typescript
- Prototype: Figma

Organisation:

- Project Management: Jira
- Kommunication: Teams Channel
- Version Control: Github

02.05.2026

UC1 – Conduct Pre-Travel Preference Chat **Epic:** Pre-Travel **Label:** pre-travel

Description: The user interacts with the AI through a step-by-step chat interface. The AI asks questions one at a time to collect all necessary trip details — origin, destination, group size, duration, budget, accommodation, food preferences, and activity interests — before generating any recommendations.

Acceptance Criteria:

- AI opens with a greeting and asks the first question (where are you traveling from?)
- AI then asks for the destination the user already has in mind
- Each question is asked one at a time
- A step progress indicator is shown (e.g. "Step 5/10")
- User can type freely or select a quick-reply chip if provided
- Alternatively, user can directly give all the information in one chat bubble
- If the user's answer is unclear, AI asks a follow-up before moving on
- All collected inputs are stored and passed to the recommendation engine after the final step

UC2 – Present Recommendation Menu **Epic:** Pre-Travel **Label:** pre-travel

Description: After the preference chat is complete, the AI summarizes the trip details it collected and presents the user with a menu of planning categories to dive into. The user picks one and the AI routes them into the relevant follow-up use case.

Acceptance Criteria:

- AI sends a summary of collected inputs for the user to confirm (e.g. "So you're traveling from Hamburg to Hanoi, 3 people, 12 days, medium budget…")
- User can correct the summary if something is wrong before proceeding
- After confirmation, AI asks what the user wants to plan first
- The menu offers at least: Hotels, Restaurants, Activities, Transportation, Full Itinerary
- User can select by typing naturally or tapping a quick-reply chip
- AI routes to the correct use case based on the selection (UC4, UC5, UC6, UC7, UC8)
- After any category is completed, AI brings the user back to the menu to pick the next one
- User can type "done" or similar to end the planning session

UC3 – Compare Alternative Destinations **Epic:** Pre-Travel **Label:** pre-travel **Is blocked by:** UC2

Description: The user can ask the AI within the chat to compare two or more of the suggested destinations, and the AI responds with a side-by-side style comparison in the conversation.

Acceptance Criteria:

- User can request a comparison by typing naturally (e.g. "compare option 1 and 2")
- AI returns a comparison covering at least: vibe, budget fit, and activity types
- Comparison appears as a readable chat message, not a separate page
- User can continue the conversation after the comparison

UC4 – Generate Full Trip Itinerary **Epic:** Pre-Travel **Label:** pre-travel, potential-feature

Description: Once a destination is chosen, the system generates a day-by-day itinerary with morning, afternoon, and evening slots filled with activities, meals, and transport suggestions.

Acceptance Criteria:

- User provides trip duration (number of days)
- System generates a structured itinerary per day
- Each day includes at least one activity, one restaurant, and one transport note
- Itinerary respects the user's budget and interest preferences
- User can request a regenerated itinerary if unsatisfied

UC5 – Get Hotel Recommendations Epic: Pre-Travel **Label:** pre-travel

Description: The system recommends hotels at the chosen destination based on the user's budget level and accommodation preference.

Acceptance Criteria:

- User can specify accommodation type (hostel, 3/4/5-star hotel)
- System returns at least 2 hotel options
- Each option includes name, price range, star rating, and a short description
- Recommendations match the selected budget level

UC6 – Get Restaurant Recommendations Epic: Pre-Travel **Label:** pre-travel

Description: The system recommends restaurants at the destination based on the user's food preferences and any dietary restrictions.

Acceptance Criteria:

- User can select food type (local, vegetarian, international, etc.)
- User can optionally specify dietary restrictions (gluten-free, nut allergy, etc.)
- System returns at least 2 restaurant options
- Each option includes name, cuisine type, and a special note if relevant (e.g. "vegan-friendly")

- Conflicting preferences (e.g. vegan + seafood) trigger a clarification prompt

UC7 – Get Activity Recommendations Epic: Pre-Travel **Label:** pre-travel

Description: The system suggests activities at the destination that match the user's selected interest categories and travel style.

Acceptance Criteria:

- System returns at least 3 activity suggestions
- Each activity includes a name and short description
- Activities match at least one of the user's selected interest categories
- Results are filtered to fit within the user's budget level

UC8 – Get Transportation Plan Epic: Pre-Travel **Label:** pre-travel

Description: The system suggests how to get to the destination and how to get around once there, based on the user's starting point and budget.

Acceptance Criteria:

- User provides a start point and destination
- System suggests at least one travel option to the destination (flight, train, or bus)
- System suggests at least one local transport option (public transport, taxi, walking)
- Suggestions are consistent with the user's budget level
- No real-time pricing is required — suggestions can be general

UC9 – Modify / Regenerate Travel Plan Epic: Pre-Travel **Label:** pre-travel

Description: The user can update their preferences and regenerate any part of the travel plan without starting from scratch.

Acceptance Criteria:

- User can change any preference (budget, interest, accommodation type)
- System regenerates only the affected recommendations
- Previously generated results are replaced with the new ones
- User receives a confirmation that the plan has been updated

UC10 – Handle Conflicting Preferences Epic: Cross-Cutting **Label:** pre-travel, potential-feature

Description: When the system detects that two or more user inputs contradict each other, it flags the conflict and asks the user to clarify before proceeding.

Acceptance Criteria:

- System detects common conflicts (e.g. luxury experience + budget level, vegan + seafood)
- User is shown a clear message explaining the conflict
- User is offered two or more resolution options to choose from
- System proceeds only after the conflict is resolved

UC11 – Log Visited Places Epic: During Travel **Label:** during-travel

Description: While traveling, the user can log places they have already visited so the system can avoid recommending them again and use the history to inform next suggestions.

Acceptance Criteria:

- User can add a place name and approximate duration of visit
- Logged places appear in a visible history list
- Already-visited places are excluded from future recommendations
- User can remove a logged place if added by mistake

UC12 – Input Current Mood Epic: During Travel **Label:** during-travel

Description: The user selects their current mood or energy state so the system can tailor the next recommendation to how they are feeling in the moment.

Acceptance Criteria:

- User can select a mood from a predefined list (e.g. energetic, tired, hungry, relaxed)
- Mood selection updates the context used for the next recommendation
- User can change their mood at any time
- If no mood is selected, system uses the last known mood or asks the user

UC13 – Get Next Attraction Recommendation Epic: During Travel **Label:** during-travel

Description: Based on the user's current mood, visited history, and location, the system recommends the single best next attraction or activity to visit.

Acceptance Criteria:

- System returns one primary recommendation with a name and short reason
- Recommendation excludes already-visited places
- Recommendation reflects the user's current mood
- User can ask for an alternative if they don't like the suggestion

UC14 – Get Context-Aware Recommendation Epic: During Travel **Label:** during-travel, potential-feature

Description: The system combines the user's location, current time, weather, mood, and visit history to compute a highly personalized next recommendation dynamically.

Acceptance Criteria:

- System uses at least 3 context signals (mood + history + one of: location, time, or weather)
- Recommendation updates when any context signal changes
- System explains briefly why this recommendation was made
- Fallback recommendation is shown if context data is incomplete

UC15 – Adapt Recommendations After Skip **Epic:** During Travel **Label:** during-travel, potential-feature

Description: If the user skips or dismisses a recommendation, the system registers this and adjusts future suggestions so it does not keep repeating similar options.

Acceptance Criteria:

- User can dismiss or skip any recommendation
- Skipped recommendations are not shown again in the same session
- System adjusts the next recommendation based on the skip
- After 3 skips in a row, system asks the user if their preferences have changed

UC16 – Get Nearby Restaurant During Travel **Epic:** During Travel **Label:** during-travel

Description: While exploring, the user can ask for a restaurant recommendation near their current location that fits their food preferences.

Acceptance Criteria:

- User can trigger a "find food nearby" action
- System returns at least one restaurant recommendation
- Recommendation respects the user's dietary preferences set earlier
- If no location is available, system asks user to input their current area manually

UC17 – Get Transport to Next Destination **Epic:** During Travel **Label:** during-travel, potential-feature

Description: The system suggests how to get from the user's current location to their next recommended attraction, using simple transport options.

Acceptance Criteria:

- System suggests at least one transport option (walk, taxi, public transport)
- Suggestion is relevant to the distance and user's budget
- No real-time API integration required — general suggestions are acceptable for v1
- User can request an alternative transport option

UC18 – Detect and Resolve Preference Conflicts Epic: Cross-Cutting **Label:** potential-feature

Description: A dedicated system layer that scans all user inputs at any point in the flow and flags combinations that would produce poor or contradictory recommendations.

Acceptance Criteria:

- System checks for conflicts on every preference update, not just at submission
- Detected conflicts are shown with a plain-language explanation
- User must resolve the conflict before the next recommendation is generated
- System logs resolved conflicts to avoid flagging the same pair twice

UC19 – Persist Travel History Across Session Epic: Cross-Cutting **Label:** potential-feature

Description: The user's visited places, preferences, and mood history are saved so the session can be resumed or referenced later without re-entering everything.

Acceptance Criteria:

- User data persists for the duration of an active session
- On return, user is shown their last known state
- User can clear their history manually
- No login required for v1 — session storage is sufficient

Kleine Präsentation:

(7-Min) Use Cases: >15 Use Cases: Amal, Rawan

(3-Min) Zusammenfassung: Min

(5-7Min) Technology:Min

(3-5 Min) Prototype: Linh

https://www.canva.com/design/DAHIgZVQAao/x-arBhw73GOW80FcpmH8ag/edit

when user wants to add their own information, not rely solely on gemini AI.